WNDRVR
SUPPORT NETWORK

VXWORKS 7 NAT PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 NAT Programmer's Guide, SR0650

14 August 2020

1. OVERVIEW

1. About Wind River NAT

Wind River Network Address Translation (Wind River NAT) is an implementation of traditional NAT. Its key feature is to translate private network addresses, which may be invalid outside the private network, into addresses recognizable to a public network such as the Internet.

The chief advantage of this feature is that addresses on the private network are hidden from the public Internet, providing a measure of security. A second advantage, realized with certain varieties of NAT, is that scarce IP addresses are conserved, reducing network administration costs.

NAT is typically used on routers and gateways that forward packets between private and public networks, such as home or small office Internet gateways.

You can develop NAT rules using a simple keyword syntax and load these rules with a rule file, the Wind River NAT API, or the nat shell command.

About the Addresses Used in Examples

According to RFC 1918, the Internet Assigned Numbers Authority (IANA) has reserved the following three blocks of the IPv4 address space for private internets:

- 10.0.0.0 - 10.255.255.255 (10/8 prefix)
- 172.16.0.0 - 172.31.255.255 (172.16/12 prefix)
- 192.168.0.0 - 192.168.255.255 (192.168/16 prefix)

These address spaces are also useful in networking examples, which need to function but also need to avoid public Internet addresses.

In this book, the 10/8 prefix, the largest of the three private address spaces, represents the public Internet. To represent a private address space, this book uses the 192.168/16 prefix.

Additional Documentation

The following Wind River documents may be helpful to you when working with Wind River NAT:

- VxWorks 7 Configuration and Build Guide
- VxWorks 7 Network Stack Programmer's Guide

The following third party documents may be helpful to you when working with Wind River NAT:

- Kumar, V., Korpi, M., and Sengodan, S. *IP Telephony with H.323: Architectures for Unified Networks and Integrated Services*. New York: John Wiley & Sons, Inc., 2001.

2. RFCs

Wind River NAT complies with the following RFCs, except where noted otherwise.

These RFCs are accessible from the following Web site: http://www.ietf.org.

Supported RFCs

- RFC 2663, *IP Network Address Translator (NAT) Terminology and Considerations*. August 1999, Srisuresh, P. and Holdrege, M.
- RFC 2766, *Network Address Translation—Protocol Translation (NAT-PT)*. February 2000, Tsirtsis, G. and Srisuresh, P.
- RFC 3022, *Traditional IP Network Address Translator (Traditional NAT)*. January 2001, Srisuresh, P. and Egevang, K.

Related RFCs

- RFC 1034, *Domain Names - Concepts and Facilities*. November 1987, Mockapetris, P.
- RFC 1035, *Domain Names - Implementation and Specification*. November 1987, Mockapetris, P.
- RFC 1701, *Generic Routing Encapsulation (GRE)*. October 1994, Hanks, S., Farinacci, D., and Traina, P.
- RFC 1886, *DNS Extensions to support IP version 6*. December 1995, Thomson, S., Huitema, C.
- RFC 1918, *Address Allocation for Private Internets*. February 1996, Rekhter, Y., Moskowitz, B., Karrenberg, D., de Groot, G. J., and Lear. E.
- RFC 2406, *IP Encapsulating Security Payload (ESP)*. November 1998, Kens, S., and Atkinson, R.
- RFC 2408, *Internet Security Association and Key Management Protocol (ISAKMP)*. November 1998, Maughan, D., Schertler, M., Schneider, M., and Turner, J.
- RFC 2428, *FTP Extensions for IPv6 and NATs*. September 1998, Allman, N., Ostermann, S., and Metz, C.

  RFC 2874, *DNS Extensions to Support IPv6 Address Aggregation and Renumbering*. July 2000, Crawford, M., Huitema, C.

3. NAT Overview

Wind River NAT supports the following features:

- Basic NAT.
- Network address port translation (NAPT).
- Bidirectional NAT.
- Network address translation-protocol translation (NAT-PT).
- NAPT-PT.
- Demilitarized zone (DMZ) host.
- Application-level gateways (ALGs).
- Port triggering.

Wind River NAT supports these NAT modes to provide a comprehensive framework for address translation and communication between private and public networks. Together, these modes optimize security and connectivity while conserving public IP addresses.

The following sections provide additional information on the Wind River implementation of each NAT mode.

Basic NAT

In basic NAT, a router or gateway connects a private network to a public network, using one public IP address for each connection to an external host. As outgoing packets from hosts on the private network pass the router, NAT replaces the source addresses in those packets with the router's public IP address. This translation, or mapping, conceals private network addresses from hosts on the public network.

NAT records the mapping of private host source addresses to public host destination addresses. When a reply arrives from a public host, NAT uses this mapping to route the reply to the correct host on the private network.

In basic NAT mode, Wind River NAT permits private hosts to initiate connections with hosts on the public Internet. These connections, which are outbound from the private network to the public, are considered unidirectional at initiation. Once initiated, such connections become bidirectional.

NAPT

NAPT extends the capabilities of basic NAT by translating the port field in outgoing packet headers in addition to the source address field. This feature allows the gateway to handle multiple simultaneous connections from multiple hosts on the private network to the same server on the public side.

A device running NAPT can connect an entire department or small office to the Internet using only a single global IP address. This feature saves network administration costs by reducing the number of public IP addresses that must be purchased or leased from a service provider.

Bidirectional NAT

In bidirectional NAT, hosts on a public network can initiate connections to hosts on the private network. Wind River NAT supports static translation entries to permit such connections.

NAT maps private network addresses to globally unique public addresses as connections are established in either direction. Public network hosts access private network hosts using DNS for address resolution. Therefore, if the name server is located on the private side of the NAT gateway, a DNS ALG is required to enable bidirectional NAT.

NAT-PT

In NAT-PT, the router is equipped with a dual TCP/IP stack so that it can translate IPv6 addresses to IPv4 addresses. This facility establishes a transparent communication path between IPv6 networks and IPv4 networks, allowing IPv6 hosts on the private network to communicate with IPv4 hosts on the public network. Address bindings are dynamic. NAT-PT operation is unidirectional from the private network to the public at initiation. Public network hosts can only respond to connections initiated from private network hosts. They cannot initiate their own connections.

NAPT-PT

NAPT-PT combines the capabilities of protocol translation with port translation to enable transparent communication between IPv6 and IPv4 hosts, using a single IPv4 address. NAT-PT translates the TCP/UDP ports of the IPv6 hosts into the TCP/UDP ports of the registered IPv4 host.

The advantage of combining protocol translation with port translation is that it makes more efficient use of the address pool available for mapping connections between IPv6 hosts and IPv4 hosts. NAPT-PT allows an IPv4 host to conduct simultaneous TCP and UDP sessions using a single IPv4 address—up to 63,000 for each protocol—rather than requiring a unique IPv4 address for each session.

DMZ Host

DMZ host functionality lets you specify a host (on the private network) to which the router forwards all packets not handled by NAT. The private network host that receives the forwarded packets is known as the DMZ host. This host, although resident on a private network, is still externally accessible to connections initiated on the external network. For more information, see Configuring a DMZ Host.

Application-Level Gateways

All NAT modes include IP address translation. NAPT also includes the translation of TCP/UDP port entries. However, some applications use IP addresses and port numbers inside their data payloads. To extend the capabilities of NAT and enable it to

operate with such applications, application-level gateways (ALGs) can modify such information within data payloads. Because different applications employ different protocols or data formats, ALGs must be customized for each application.

Wind River NAT includes the ALG software for the following protocols and applications:

- DNS
- FTP.
- H.323.
- ICMP
- IPsec passthrough.
- PPTP passthrough.
- Session initiation protocol (SIP).
- Port triggering.

ICMP is built into Wind River NAT. You can also create additional ALGs. For more information, see Application-Level Gateways - Overview.

Port Triggering

Port triggering lets you dynamically open inbound ports to external connections based on outbound traffic. For information on port triggering, see Port Triggering on page 31.

API for Integrating a Custom ALG with Wind River NAT

Wind River NAT includes a set of API routines you can use to integrate a custom ALG with NAT. Using this API, your ALG can create NAT mappings to let incoming traffic through, modify application-specific payloads, and do whatever is needed to get the application running across disparate address realms.

RFC 3022 Checksum Adjustment

Each time Wind River NAT makes an address and port translation, it adjusts the checksums in the IP header and in the TCP/UDP headers. To minimize overhead, the checksum adjustment is made according to the checksum adjustment algorithm suggested in RFC 3022, 4.2 Checksum Adjustment, rather than calculating the checksums from scratch.

Configuration Interfaces

Wind River NAT provides the following configuration interfaces:

- APIs
- shell command

API Library and Shell Command

The public API library contains utilities for the translation of addresses, ports, and protocols. These routines are useful for testing and debugging. The nat shell command provides access to the same functionality.

2. CONFIGURATION AND BUILD

1. Configuration and Build Workflow

Your VxWorks 7 installation includes source code but does not include pre-built kernel libraries. Before beginning any VxWorks development, you must build the VxWorks kernel libraries. You must also build a VxWorks run-time kernel image based on the kernel libraries.

For specific instructions on how to configure and build a VSB project and a VIP, and how to create a downloadable kernel module (DKM) or a real-time process (RTP) application, see the *VxWorks 7 Configuration and Build Guide*.

For a list of available source configuration options for the Wind River NAT, see Source Build Configuration Options (VSB on page 5.

For a list of available kernel configuration components for Wind River NAT, see Kernel Configuration Components (VIP) on page 6.

2. Source Build Configuration Options (VSB)

NAT support is delivered as a part of the core networking RPM and is included as part of the core networking layer.

NAT VSB Options table on page 5 lists the NAT VSB options.

Table 1. NAT VSB Options

Option	Description
Enable IPNET Network Address Translation (COMPONENT_NAT)	Enables NAT support in the VSB.

Workbench 4

For Workbench 4, the NAT VSB option is under **net > Enable IP Networking Stack > Enable Networking Stack Core > IPNET_COREIP Menu > Layer 3 (Network) Components**.

To locate NAT options in the Source Build Configuration editor, use the **Find** utility and enter **NAT** as the search string.

Note: The NAT VSB configuration option is an attribute of the networking stack core VSB configuration option. The core networking stack option (**IPNET_COREIP**) must be enabled in order to enable NAT.

vxprj

wrtool does not support interactively creating or configuring VSBs. This workflow uses vxprj instead. For vxprj, you must enable the networking stack core VSB option before you can enable NAT.

Step 1: Navigate to your vsb project directory.

Step 2: Enable the networking stack core VSB configuration option. You can list the networking stack core option as follows:

- To list the *available* core networking options in the VSB, use the prj vsb listAll command. For example:

```
projectDir> vxprj vsb listAll
   ...
   IPNET_1_0_0_0_COREIP_1_0_0_0
```

- To list the enabled core networking options in your project, use the prj vsb list command. For example:

```
projectDir> vxprj vsb list
   ...
   IPNET_1_0_0_0_COREIP_1_0_0_0
```

To add the networking stack core VSB option to your project, use the prj vsb add command. For example:

```
projectDir> vxprj vsb add IPNET_1_0_0_0_COREIP_1_0_0_0
```

To remove networking stack core VSB options from your project, use the prj vsb remove command. For example:

```
projectDir> vxprj vsb remove IPNET_1_0_0_0_COREIP_1_0_0_0
```

Step 3: Enable the NAT VSB configuration attribute using the vxprj -add flag as follows:

```
projectDir> vxprj vsb config -s -add "_WRS_CONFIG_COMPONENT_NAT=y"
```

To remove the NAT VSB option from your project, use the prj -remove flag as follows:

```
projectDir> vxprj vsb config -s -remove "_WRS_CONFIG_COMPONENT_NAT=y"
```

Note that this step requires **vxprj. wrtool** does not support creating or configuring VSBs interactively.

3. Kernel Configuration Components (VIP)

The VIP component required to enable NAT functionality in your kernel image is **NAT (INCLUDE_IPNET_USE_NAT)**.

For information on optional kernel configuration components related to NAT, see Optional Kernel Configuration Components on page 8. For information on setting the NAT configuration parameters, see Nat Configuration Parameters on page 7.

Workbench 4

For Workbench 4, the NAT VIP components are under **Network Components > Layer 3 (Network) Components > NAT**.

To locate NAT components and parameters in the **Kernel Configuration** editor, use the **Find** utility and enter NAT as the search string.

wrtool

For **wrtool**, you can list the NAT VIP components as follows:

- To list the *available* NAT components for the VIP, use the following command:

  ```
  projectDir> prj component list all NAT
  ```

- To list the *included* FTP components in your VIP, use the following command:

  ```
  projectDir> prj component list NAT
  ```

To add NAT components to your VIP, use the following command syntax:

```
projectDir> prj component add componentName
```

To remove NAT components from your VIP, use the following command syntax:

```
projectDir> prj component remove componentName
```

4. NAT Configuration Parameters

NAT Configuration Parameters on page 7 lists the NAT configuration parameters.

Table 1. NAT Configuration Parameters

Workbench 4 Description, Parameter, System Variable, and Description	Default and Type
auto port end interval IPNAT_AUTOPORT_END_INTERVAL ipnet.nat.autoport.start End of the interval used for automatically generated NAT ports.	"**29999**" char *
auto port start interval IPNAT_AUTOPORT_START_INTERVAL ipnet.nat.autoport.end Start of the interval used for automatically generated NAT ports.	"**29000** " char *
icmp mapping timeout IPNAT_ICMP_MAPPING_TIMEOUT ipnet.nat.timeout.icmp Specifies the timeout in seconds until an ICMP mapping expires.	"**10** " char *
max active mapping IPNAT_MAX_MAPPING ipnet.nat.capacity.mappings The maximum number of active NAT mappings.	"**1000** " char *

Workbench 4 Description, Parameter, System Variable, and Description	Default and Type
other mapping timeout IPNAT_OTHER_MAPPING_TIMEOUT ipnet.nat.timeout.other Specifies the timeout in seconds until other mappings from protocols other than ICMP, UDP, and TCP expire.	"**60** " char *
tcp mapping timeout IPNAT_TCP_MAPPING_TIMEOUT ipnet.nat.timeout.tcp Specifies the timeout in seconds until a TCP mapping expires.	"**432000** " char *
udp mapping timeout IPNAT_UDP_MAPPING_TIMEOUT ipnet.nat.timeout.udp Specifies the timeout in seconds until a UDP mapping expires.	"**60** " char *

5. Optional Kernel Configuration Components

To include the nat shell command in your project, include **INCLUDE_IPNAT_CMD** in your VxWorks image project.

This command is only required if you want to want to configure NAT with the shell command. The nat shell command requires a file system to be included in your project.

6. Wind River NAT and Symmetric Multiprocessing

If you build Wind River NAT for use on a target configured with symmetric multiprocessing (SMP), the SMP capability of NAT is automatically enabled.When enabled, the NAT hooks run in parallel on multiple cores which results in improved performance.

7. Adding a Hook for NAT Rules

If you plan to add NAT rules at startup by calling ipnet_nat_add_rule( ), add a hook for those rules.

To create this hook, add a **USER_APPL_INIT** macro in the BSP. For example:

```
#define INCLUDE_USER_APPL
#define USER_APPL_INIT \
{ \
IMPORT void usrNATAddRules();\
usrNATAddRules();\
}
```

In this example, usrNATAddRules( )is a sample routine only. It is not distributed with your product installation. You must create it (or a routine with a similar name) yourself.

Note: Some BSPs include sample definitions of **INCLUDE_USER_APPL and USER_APPL_INIT**. If so, remove those examples. Define **INCLUDE_USER_APPL and USER_APPL_INIT** only once.

WNDRVR

3. NAT TUTORIAL

1. Overview

This chapter contains a tutorial that will guide you in implementing NAT. The tutorial provides information on writing NAT rules and testing the NAT system. It also provides information on using Workbench 4 to develop and deploy NAT.

2. Network Configuration

The NAT system created in this tutorial is designed to run on a simple network consisting of the following nodes:

- A public host.
- Two private hosts.
- A gateway with two interfaces.
- A switch (optional).

NAT Configuration Parameters on page 10 provides configuration information for each node.

Table 1. NAT Configuration Parameters

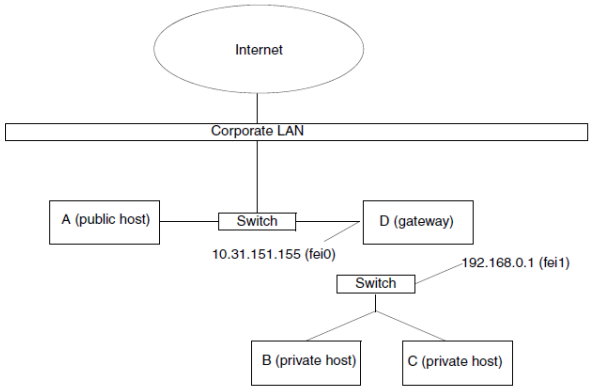
Node	IP Address	Required Software
A (public host)	10.31.100.21	**FTP server** **Web server** **Web browser** **Ping command**
B (private host)	192.168.0.2	**FTP client** **Web server** **Web browser** **Ping command** **Telnet client command**
C (private host)	192.168.0.3	**FTP client** **Web server** **Web browser** **Ping command** **Telnet client command (optional)**

Node	IP Address	Required Software
D (gateway)	**0.31.151.155 on fei0 (public interface)** **192.168.0.1 on fei1 (private interface)**	

Note: Hosts B and C must be configured with a route to the 10.31.0.0/16 network via 192.168.0.1. Host A must not be configured with a route to the 192.168.0.0/24 network.

If desired, this network can also be connected to a corporate LAN and, through that LAN, to the Internet. Tutorial Network Configuration on page 11 illustrates this configuration.

Figure 1. Tutorial Network Configuration



3. Implementing NAT

This tutorial explains how to create a simple gateway using NAT rules.

Note: This tutorial assumes that you have already connected the required hardware, created a VSB and a VIP based on the VSB, and added a hook for NAT rules (see Adding a Hook for NAT Rules on page 8). If you have not already performed these tasks, do so now. For further information, see Network Configuration on page 10.

NAT Rules

The rule set in this tutorial implements the following policy:

- An FTP ALG called when a TCP packet crosses port 21.
- NAPT for TCP and UDP packets, using a source port in the 18000:18999 interval for translation.
- NAPT for ICMP packets, using a source port in the 19000:19999 interval for translation.
- Basic NAPT for other protocols.

Note the order in which these features are implemented. The first rule enables the FTP ALG to ensure that this rule is parsed before the packet matches an address or port specification in a NAT, NAT-PT, or NAPT rule. (For further information, see NAT Operation on page 15.)

Writing Rules

Step 1: Implement the FTP ALG

Create a rule that implements the FTP ALG. To create this rule, call the ipnet_nat_add_rule( ) routine, using the appropriate keywords as parameters. Use the following routine:

```
ipnet_nat_add_rule("map fei0 0/0 to 0/32 proxy port 21 ftp/tcp"); \
```

Note: To include the ALG in your VxWorks image, enable the macro #define **IPNET_USE_NAT_FTP_ALG** in the following file:*installDir* **/vxworks-7/pkgs_v2/net/ipnet/coreip-**x.x.x.x **/src/ipnet2/config/ ipnet_config.h**

Step 2: Implement NAPT for TCP and UDP Packets

Create a rule that implements NAPT for TCP and UDP packets. Use the following routine:

```
ipnet_nat_add_rule("map fei0 0/0 to 0/32 portmap tcp/udp 18000:18999"); \
```

Step 3: Implement NAPT for ICMP Packets

Create a rule that implements NAPT for ICMP packets. Use the following routine:

```
ipnet_nat_add_rule("map fei0 0/0 to 0/32 icmpidmap icmp 19000:19999"); \
```

Step 4: Implement Basic NAPT for Other Protocols

Create a rule that implements basic NAPT for other protocols. Use the following routine:

```
ipnet_nat_add_rule("map fei0 0/0 to 0/32"); \
```

When complete, the NAT code should look something like the following:

Complete NAT Code

```
/*
DESCRIPTION
Initialize user application code.
*/

#include <vxWorks.h>
#if defined(PRJ_BUILD)
#include "prjParams.h"
#endif /* defined PRJ_BUILD */

#ifndef INCLUDE_USER_APPL
#define INCLUDE_USER_APPL
#endif

/* Example NAT ruleset for a Home/SOHO gateway. The internal network is on
'fei1' and the external on 'fei0'. The ruleset enables NAPT for TCP, UDP, and
```

```
ICMP request/reply and basic NAT for other protocols. The first rule enables the
FTP ALG. No services are available on any internal host. */

#define USER_APPL_INIT { \
    IMPORT int ipnet_nat_add_rule(const char *rule); \
    /* Enable the FTP ALG */ \
    ipnet_nat_add_rule("map fei0 0/0 to 0/32 proxy port 21 ftp/tcp"); \
    /* Enable NAPT for TCP and UDP */ \
    ipnet_nat_add_rule("map fei0 0/0 to 0/32 portmap tcp/udp 18000:18999"); \
    /* Enable NAPT for ICMP */ \
    ipnet_nat_add_rule("map fei0 0/0 to 0/32 icmpidmap icmp 19000:19999"); \
    /* Enable basic NAPT for other protocols */ \
    ipnet_nat_add_rule("map fei0 0/0 to 0/32"); \
    }

/*******************************************************************************
*
* usrAppInit - initialize the users application
*/

void usrAppInit (void)
    {
#ifdef     USER_APPL_INIT
           USER_APPL_INIT; /* for backwards compatibility */
#endif
    }
```

Testing the NAT Implementation

To test the NAT gateway to verify that it is working, do the following:

1. Begin by disabling NAT. To do so, issue the following command from a target shell:

   # **nat -D**

   > Note: To run this command, first switch to the command interpreter shell, then type cmd at the command prompt. Run the nat command.

2. Perform the following tests:
   - Ping from B to A.
   - Ping from C to A.

   With NAT disabled and no direct route connecting host A to the private 192.168.0/0 network, these tests should fail.
3. Enable NAT by issuing the following command from a target shell:

   # **nat -E**

4. Perform the following tests:
   - Ping from B to A.
   - Ping from C to A.

   With NAT enabled, these pings should succeed.

5 Perform the following additional tests:

- Web browsing from B to A.
- Web browsing from C to A.
- FTP from B to A (with FTP in active mode).
- FTP from C to A (with FTP in active mode).

These tests should succeed.

6. Check NAT statistics by issuing the following shell command:

```
[vxWorks *] # nat -s
```

Translated packets in and out, added mappings, and expired mappings should all be greater than 0. The NAT statistics look similar to the following:

```
translated: in 87 out 45
     nomatch: in 0 out 0
     invalid: in 0 out 0
     dropped: in 0 out 0
     added mappings: 14
     expired mappings: 14
     failed mappings: 0
```

4. NAT FUNDAMENTALS

1. Overview

This chapter describes core NAT concepts, including the elements of a NAT rule, the processing of rules, and different methods for writing rules. It also describes how to configure NAT to operate in different modes and how to configure a DMZ host.

2. NAT Operation

Wind River NAT is designed for use on routers and gateways that forward packets between private networks and a public network, such as the Internet.

NAT relies on the normal forwarding mechanism in the TCP/IP stack. Packets received from the private network are forwarded by the stack and then intercepted by NAT for translation before being handed over to the driver. Similarly, packets received from the Internet are intercepted by NAT for translation and then forwarded to the private network.

You can save NAT rules in a text file, which you can store anywhere on the system. NAT parses the rules from the top down and stops parsing when it finds a matching rule. Therefore, it is important that rules are added in the right order to ensure complete processing of all packets.

For optimal results, arrange rules in the following order:

1. ALG rules.
2. NAPT rules.
3. Basic NAT or NAT-PT rules.
4. Bidirectional NAT rules.

Outbound Packets

NAT processes outbound packets by translating their source addresses, protocols, and ports, as applicable, and recording the translations for subsequent use in the processing of inbound packets. The following sections provide additional details on the various NAT modes.

NAT and NAPT Operation

NAT compares outbound packets from the private network with the basic NAT and NAPT rules in its rule set. If a packet matches a rule, NAT translates the source address as specified in the rule. If the rule also specifies port translation, NAT translates the port as well.

When the packet is translated, NAT records, or maps, the combination of source and destination addresses, protocol, and ports for future use. If the protocol is UDP, TCP, or ICMP echo request, NAT also maps the source and destination port or the echo request identifier. Subsequent outbound packets are checked first against such mappings, then against the NAT and NAPT rules. Mappings are also used for the subsequent translation and routing of inbound packets received in response to outbound packets.

NAT-PT and NAPT-PT Operation

NAT-PT intercepts IPv6 packets outbound from the private network and compares them with the NAT-PT rules in its rule set. If a packet matches a rule, its address is translated to IPv4 format and forwarded through the router to the Internet. NAT-PT records the combination of addresses for future use and marks the mapping as NAT-PT-originated. This mapping is used for the subsequent translation and routing of inbound packets received in response to outbound packets.

Note:

To connect to IPv4 hosts on the Internet, IPv6 hosts on a private network must be configured to route their packets to the PREFIX::/96 network. Alternately, this configuration can be applied to the NAT-PT router.

The PREFIX network is an arbitrarily chosen bogus network. If a user on the IPv6 network wishes to ping the host 192.42.198.5, the following command is required:

```
# ping PREFIX::192.42.198.5
```

For further information on NAT-PT, see RFC 2766.

Handling of Fragments

Fragmented outbound packets can cause problems for NAT gateways. (Incoming packets from the outside of the NAT do not cause any problems because the router reassembles packets before applying NAT rules.)

Fragmented packets coming from the private network can cause problems if the the fragments arrive out of order. If the second fragment arrives before the first, it cannot be matched against NAPT rules and may be translated incorrectly. However, most TCP/IP stacks send the fragments in order and there are typically no routers on the private network that can cause the fragments to be lost or reordered. When the first fragment transmitted is the first to arrive, a mapping is added, which can then be used by the second fragment. All fragments are thereby translated correctly.

Fragments can also cause problems for ALGs, because the ALG routine does not have access to the complete packet at the same time. When the routine cannot access the complete packet, problems may occur in checksum recalculation. For this reason, ALGs cannot change the application data for fragmented packets. Also, if the second fragment arrives before the first, the ALG will never be called.

The Wind River NAT-PT implementation has some additional limitations for fragments:

- Fragmented TCP packets and ICMP echo requests are not translated.
- UDP packets are translated with a zero checksum, which is allowed for IPv4 UDP.

Inbound Packets

Inbound packets from the Internet receive slightly different processing. NAT compares the packet against its active NAT and NAPT mappings to determine if the packet should be forwarded to a host on the private network or to the NAT router itself. If NAT finds a relevant mapping, it translates the packet's destination address and port to the address and port of the specified private host. If the packet does not match any active mappings, NAT compares it with the bidirectional NAT rules in its rule set. If NAT finds a matching bidirectional rule, it maps the combination of public source and private destination addresses—and ports, if applicable—for future use, then performs the appropriate translations and forwards the packet to the specified private network host.

DMZ Host

A DMZ host is a computer on the private network to which the router forwards all packets not handled by NAT. This host, although resident on a private network, is still externally accessible to connections initiated on the external network.

Preserving Access to Services on a Gateway Running a DMZ Host

If the NAT rule set designates a DMZ host, all normal service requests are forwarded to that host. As a result, all services normally available on the gateway (such as an FTP server, web server, or telnet) are effectively disabled for the gateway. If you need to preserve the availability of these services on the gateway, you must add bidirectional NAT rules that redirect the services to the gateway. These rules must be added before the DMZ host rule. Otherwise, the DMZ host rule overrides the redirection rules. For further information and sample rules, see Configuring a DMZ Host on page 21.

ICMP Requests to a Gateway Running a DMZ Host

Incoming echo requests (pings) to the NAT router are normally sent up to the router's host stack. However, if NAT has been configured with a DMZ host rule, the ping is sent through the router to the DMZ host. You can override this behavior by placing a

bidirectional NAT rule before the DMZ host rule that redirects the echo request to a specific host on the private network or to the NAT router itself. For further information and sample rules, see Configuring a DMZ Host on page 21.

3. Elements of a NAT Rule

Each NAT rule consists of the following elements:

- An action to be taken, such as map, rdr (redirect), or pt (protocol translation).
- A network interface specification.
- A pair of addresses for translation.
- The keyword to to join the translated addresses.
- A port specification.

The first four parameters are required. The last is optional.

This practice is useful with shells like bash and others that use the > character for other purposes.

4. Methods for Writing Rules

You can develop NAT rules using a simple keyword syntax and load these rules with a rule file, the Wind River NAT API, or the nat shell command.

For further information on adding rules to the active rule set, see Adding and Removing NAT Rules on page 21.

Using a Rule File

You can write NAT rules using the keyword syntax shown in the examples in this document. Save your NAT rules in a text file and store them wherever you like on the system. You can use any file name or extension, but it is common to use a **.cfg** extension. The file must reside on the target and be stored on local media.

Empty lines and white space are permitted in a keyword file. The pound sign (**#**) precedes a comment. You can terminate a line with a comment.

The following example shows a comment line, an empty line, a line with a rule terminated by a comment, and a line with a rule. The entire file consists of four lines.

```
# example NAT rule file
map fei0 0/0 to 0/32 # use basic NAT for other protocols
rdr fei0 195.42.198.1 port 0 to 10.0.0.1 port 0 ip
For further information on a particular keyword, see its reference
```

For further information on a particular keyword, see its reference entry in About the NAT Shell Command on page 45.

Using the API

You can also create NAT rules using the Wind River NAT API. All rules should be added to the **usrAppInit.c** file in your VxWorks image project (VIP). For a description of the syntax and available parameters, see the reference entry for each routine.

Using a Shell Command

You can also create NAT rules using the nat shell command. For a description of all available parameters, see NAT Shell Command Options on page 45.

5. Configuring Basic NAT

Use map rules to configure basic NAT. For example:

```
map fei0 192.168.1.0/24 to 195.42.198.5
map fei0 10.0.0.0/8 to 0/32
map fei0 0/0 to 0/32
map fei0 0/0 to 0/32 dstmap 192.168.0.2
```

The first rule pertains to all packets going out on interface **fei0** from any source address in the 192.168.1.0/24 address space. The rule instructs NAT to replace the source address in such packets with the address 195.42.198.5.

The second rule also pertains to all packets going out on interface **fei0** from any source address in the 10.0.0.0/8 address space. The rule instructs NAT to replace the source address in such packets with the interface address fei0. The parameter 0/32 instructs NAT to use the address of the interface the packet is sent on as the new source address.

The third rule also pertains to all packets going out on interface **fei0** from any source address. The rule instructs NAT to replace the source address in such packets with the interface address of **fei0**. The parameter 0/0 makes this rule applicable to packets from any source address.

The fourth rule also pertains to all packets going out on interface fei0 from any source address. The rule instructs NAT to replace the source address in such packets with the interface address of **fei0**. The parameter 0/0 makes this rule applicable to packets from any source address. Also, the rule instructs NAT to replace the **dst** address in such packets with 192.168.0.2.

Basic NAT Limitation

The drawback with basic NAT is that if several hosts on the private network connect to the same host on the public network, there is a risk that responses from the public host will be misrouted to incorrect hosts. This error can occur if private network hosts choose the same source port, which prevents NAT from distinguishing between responses intended for different private network hosts.

There are two ways to avoid this error. The first is to use NAPT, as described in Configuring NAPT on page 19. The second is to use a new source address for each new private host whose address is being translated. The second solution, however, requires the NAT gateway to have access to many public IP addresses that it can use as alias addresses on the same interface.

Mapping Between Address Blocks

You can map between address blocks, using the map-block keyword to assign each private address a unique public address.

```
map-block fei0 192.168.1.0/24 to 195.42.198.0/24
```

This rule instructs NAT to translate source address 192.168.1.1 to 195.42.198.1, 192.168.1.2 to 195.42.198.2, and so forth. Note that with map-block rules, the network masks must be in parallel form on both sides of the **to** string for the rule to be accepted.

6. Configuring NAPT

Use portmap rules to configure NAPT. They begin with the same syntax as map rules but use the **portmap** keyword to specify additional parameters. For example:

```
map fei0 10.0.0.0/8 to 0/32 portmap tcp/udp 9000:10000
```

The rule above pertains to all packets going out on interface fei0 with a source address matching the 10.0.0.0/8 address space. The rule instructs NAT to perform the following two actions on such packets:

- Replace the source address with the interface address of fei0.
- Replace the source port with a port between 9000 and 10000.

This rule is valid for all TCP/UDP packets. You can also write a similar rule for only TCP or UDP packets, using different port intervals for each protocol. For example:

```
map fei0 0/0 to 0/32 portmap tcp 9000:10000
map fei0 0/0 to 0/32 portmap udp 19000:20000
```

If you do not want to assign a range of ports, you could configure auto to let the system automatically assign a port for you. For example:

```
map fei1 0/0 to 0/32 portmap tcp auto:5000
map fei0 0/0 to 0/32 portmap udp auto:5000
```

In the above example, "auto:5000" gives you a range of 5000 ports, where the system decides what the start and ending ports will be.

For ICMP echo requests/replies, the icmpidmap keyword instructs NAT to translate the port, based on the identifier field of the ICMP echo header. This keyword uses similar syntax as portmap but must specify ICMP as the protocol.

For example:

```
map fei0 10.0.0.0/8 to 0/32 icmpidmap icmp 20000:21000
```

Note that NAPT can only be used with TCP, UDP, and ICMP echo requests. If you want to translate other protocols, the rule set must also include a basic NAT rule after the NAPT rules. For a program example, see Sample Rule Set—Simple NAT Router.

7. Configuring Bidirectional NAT

Use redirect rules (with the rdr keyword) to configure bidirectional NAT. For example:

```
rdr fei0 195.42.198.1 port 80 to 10.0.0.1 port 8080 tcp
```

This rule instructs NAT to redirect all TCP packets arriving on interface fei0 with the destination address 195.42.198.1 and destination port 80 to a private host with address 10.0.0.1 at port 8080. To a host on the Internet, it looks like a web server is running on the NAT gateway, but instead all traffic to the web server is redirected to the private host on port 8080. A bidirectional NAT rule can also specify **udp** for UDP packets or **tcp/udp** to cover both UDP and TCP. There must be one redirection rule for service redirected to a host on the private network.

If protocols other than TCP and UDP must be redirected, there must be redirection rules for each of those protocols. In such cases, the ports are set to zero. The following example shows how to set a rule that redirects GRE packets to a private network host:

```
rdr fei0 195.42.198.1 port 0 to 10.0.0.1 port 0 gre
```

Instead of the protocol name, the protocol number can also be used:

```
rdr fei0 195.42.198.1 port 0 to 10.0.0.1 port 0 47
```

8. Configuring NAT-PT

Configuring NAT-PT is much like configuring traditional NAT. The keywords that correspond to the traditional NAT map and map-block are pt and pt-block. However, there are some important differences that apply to NAT-PT rules:

- First, the rule must specify the incoming interface on the IPv6 side of NAT-PT, because that is where NAT-PT intercepts the IPv6 packets and converts them to IPv4 packets.
- Second, the source address must be an IPv6 address or an IPv6 prefix of up to 128 bits.
- Third, it is not possible for a NAT-PT rule to specify an outgoing interface address such as 0/32 as the new source address. The NAT-PT rule must explicitly specify a new source address.

The following examples demonstrate the NAT-PT configuration:

```
pt fei1 3fff::0/120 to 195.42.198.5
pt fei1 ::/0 to 195.42.198.5
pt-block fei1 3fff::0/120 to 195.42.198.0/24
```

All three rules above configure basic NAT-PT. The first rule instructs NAT to forward all IPv6 packets coming in on interface **fei1** with a source address of 3fff::0 to 3fff::ff to the IPv4 network. The new source address for the IPv4 packet will be 195.42.198.5.

The second rule is the same as the first, except that the ::/0 parameter makes this rule valid for all incoming IPv6 packets, regardless of their source address. The third rule works like a map-block rule. An incoming packet with address 3fff::1 will get the IPv4 address 195.42.198.1, 3fff::2 will get the address 195.42.198.2 and so forth.

Note: Wind River NAT does not support bidirectional NAT-PT.

9. Configuring NAPT-PT

NAPT-PT is similar to NAT-PT. Use the pt keyword in conjunction with portmap and icmpidmap, as shown in the following examples:

```
pt fei1 ::/0 to 195.42.198.5 portmap tcp/udp 9000:10000
pt fei1 ::/0 to 195.42.198.5 icmpidmap icmp 20000:21000
```

10. Sample Rule Set—Simple NAT Router

The following example implements a rule set that covers the general requirements for a NAT router.

```
map fei0 0/0 to 0/32 portmap tcp/udp 18000:18999
map fei0 0/0 to 0/32 icmpidmap icmp 19000:19999
map fei0 0/0 to 0/32
```

The first rule enables NAPT for the TCP and UDP protocols. The second rule enables ICMP echo requests. The third sets a basic NAT rule for other protocols.

11. Configuring a DMZ Host

To enable DMZ host support, use a bidirectional NAT rule. The DMZ host rule must be the last of all rules in the list because it comes into operation only when no other rules match.

The following example shows a DMZ host rule that forwards all incoming packets not handled by other NAT rules or mappings to the private host with IP address 10.0.0.1. The ports are set to zero in a DMZ host rule, and the protocol ip is used to specify that the rule is valid for all IP protocols.

```
rdr fei0 195.42.198.1 port 0 to 10.0.0.1 port 0 ip
```

You may want to preserve some services on the NAT gateway even if there is a DMZ host rule. You can do so by placing a redirect rule before the DMZ host that redirects a port to the NAT gateway itself. The following example shows how this is done:

```
rdr fei0 195.42.198.1 port 22 to 195.42.198.1 port 22 tcp
rdr fei0 195.42.198.1 port 0 to 10.0.0.1 port 0 ip
```

The first rule above preserves access to the NAT gateway's SSH server from the public Internet. The second rule redirects all other combinations of protocol and port to the DMZ host. Note that when redirection is made to the NAT gateway itself, it is not possible to redirect to a different port.

You can also choose to let the NAT gateway respond to echo requests even if there is a DMZ host rule. This can be done by placing a redirection rule for the ICMP protocol before the DMZ host rule as shown below:

```
rdr fei0 195.42.198.1 port 22 to 195.42.198.1 port 22 tcp
rdr fei0 195.42.198.1 port 0 to 195.42.198.1 port 0 icmp
rdr fei0 195.42.198.1 port 0 to 10.0.0.1 port 0 ip
```

12. Enabling and Disabling NAT

To enable NAT, type the following shell command:

```
-> cmd

[vxWorks *]# nat -E
```

To disable NAT, type the following shell command:

```
[vxWorks *]# nat -D
```

For more information on the nat shell command, see NAT Shell Command Options on page 45.

13. Adding and Removing NAT Rules

There are three ways to add rules to a NAT:

- By storing the rules in a file and loading the rule set with the nat shell command (see Saving and Restoring NAT Rules).
- By adding individual rules or an entire rule set with the nat shell command.
- By adding individual rules or an entire rule set with the Wind River NAT API.

Once added, rules are stored in an internal table in system memory. By default, NAT appends all rules to this rule set without checking for duplicates or conflicts. If you add rules by shell command or API, NAT automatically appends them to any rule set that was automatically loaded on startup.

Adding Rules

To add a NAT rule, type the following shell command:

```
[vxWorks *]# nat rule
```

where rule is the NAT rule you wish to add. The default operation is to append the specified rule to rule set. For example:

```
[vxWorks *]# nat map fei0 192.168.1.0/24 to 195.42.198.5
```

This example makes **map fei0 192.168.1.0/24 to 195.42.198.5** the last rule in an existing NAT rule set.

Adding Rules from a File

You can also store NAT rules in a text file with a name such as myrules.cfg. To load all rules in this file at once, specify the file name and path of the rules file. For example:

```
[vxWorks *]# nat -f myrules.cfg
```

If you do not specify a path, Wind River NAT tries to open the file in the current working directory. If the file is in a different directory, you can specify the absolute path to it. For example:

```
[vxWorks *]# nat -f /usr/local/ipnat/config/myrules.cfg
```

Specifying the Rule Position

By default, new rules are appended to the existing rule list. However, you can insert a rule into a list at a specified location by using the nat shell command with the index parameter (@). The index parameter must be the first parameter in the shell command. In a set of rules, the index begins with 1. For example:

```
[vxWorks *]# nat map @2 fei0 0/0 to 0/32 icmpidmap icmp 19000:19999
```

This example inserts the rule **map fei- 0/0 to 0/32 icmpidmap icmp 19000:19999** as the second rule in an existing rule set. If the initial rule set is as follows:

```
    map fei0 0/0 to 0/32 portmap tcp/udp 18000:18999
    map fei0 0/0 to 0/32
```

The resulting rule set is as follows:

```
    map fei0 0/0 to 0/32 portmap tcp/udp  18000:18999
    map fei0 0/0 to 0/32 icmpidmap icmp  19000:19999
    map fei0 0/0 to 0/32
```

Removing Rules

To remove a NAT rule, type:

```
[vxWorks *]# nat -r rule
```

To remove all NAT rules and active mappings at once, type:

```
[vxWorks *]# nat -Fr
```

Clearing Active Mappings

To clear active NAT mappings, type:

```
[vxWorks *]# nat -C
```

Checking Syntax

To check the rule syntax, type:

```
[vxWorks *]# nat -n rule
```

The -n option parses the rule syntax and reports any errors without adding the rule. When the rule syntax is correct, there is no output. When the rule syntax is incorrect, Wind River NAT reports the error. For example:

```
[vxWorks *]# nat -f mapp fei0 192.168.1.0/24 to 195.42.198.5
```

Returns the following error message:

```
Unknown action: mapp.
```

14. Saving and Restoring NAT Rules

Wind River NAT supports nonvolatile (NV) storage to the file system.To implement NV storage, save your rule set in a text file. You can use any file name or extension, but it is common to use a **.cfg** extension. The file must reside on the target and be stored on local media.

You can load these rules by specifying a nat shell command as follows:

```
-> cmd
[vxWorks *]# nat -f myrules.cfg
```

15. Viewing NAT Information

This section describes methods for displaying NAT rules, address mappings, and statistics.

Viewing Rules and Active Mappings

To view the current NAT rule set, type:

```
[vxWorks *]# nat -l
```

The following is an example of a NAT rule set:

```
NAT RULE TABLE:
   @1 map fei0 0/0 to 0/32 proxy port ftp ftp/tcp
   @2 map fei0 0/0 to 0/32 portmap tcp/udp 18000:18999
   @3 map fei0 0/0 to 0/32 icmpidmap icmp 19000:19999
   @4 map fei0 0/0 to 0/32
```

To view the currently active NAT mappings, type:

```
[vxWorks *]# nat -m
```

The following is an example of a NAT mappings table:

```
NAT MAPPINGS TABLE:
   10.50.1.1:49236 to 10.50.2.3:21 proto tcp, (10.50.2.1:29000), state EST/EST, expire in 431999
s seq_start=0, curr_delta=0, prev_delta=0
   10.50.1.2:49236 to 10.50.2.3:21 proto tcp, (10.50.2.1:29001), state EST/EST, expire in 431999
s seq_start=0, curr_delta=0, prev_delta=0
```

To clear the active NAT mappings, type:

```
[vxWorks *]# nat -C
```

To clear the all rules and mappings, type:

```
[vxWorks *]# nat -F
```

Viewing and Clearing NAT Statistics

Wind River NAT keeps the following statistics:

- number of translated packets
- number of packets not matching any NAT rule
- number of invalid packets received by NAT
- number of packets dropped by NAT
- number of mappings added by NAT
- number of mappings that expired due to timeout
- number of mappings that could not be added

To display these statistics, type the following shell command:

```
[vxWorks *]# nat -s
```

The following is an example of statistics kept by NAT:

```
NAT STATISTICS:
   translated: in 0 out 0
   nomatch: in 307 out 0
   invalid: in 0 out 0
   dropped: in 0 out 0
   added mappings: 0
   expired mappings: 0
   failed mappings: 0
```

To clear the statistics, type:

```
[vxWorks *]# nat -Z
```

5. APPLICATION-LEVEL GATEWAYS

1. Overview

All NAT modes include IP address translation.

NAPT also includes the translation of TCP/UDP port entries. However, some applications use IP addresses and port numbers inside their data payloads. To extend the capabilities of NAT and enable it to operate with such applications, application-level gateways (ALGs) modify such information within data payloads. Because different applications employ different protocols or data formats, ALGs must be customized for each application.

Wind River NAT includes the ALG software for the following protocols and applications:

- DNS
- FTP
- H.323
- ICMP
- IPsec Passthrough
- PPTP Passthrough
- port triggering

ICMP is built into Wind River NAT. You can also create additional ALGs, using the proxy keyword and #define commands to enable the appropriate macros.

API for Integrating a Custom ALG with Wind River NAT

Wind River NAT includes a set of routines you can use to integrate a custom ALG with Wind River NAT. Using this API, your ALG can create NAT mappings and modify application-specific payloads in order to enable the application to run across the gateway. For further information on using these APIs, see Writing the ALG Routine on page 33.

2. Configuring ALG Support

The keyword used to specify a NAT rule with an ALG is proxy.It is important that proxy rules are specified before other NAT rules to ensure that these rules are parsed before the packet matches an address or port specification in a NAT, NAT-PT, or NAPT rule.

The following example shows a proxy rule for the FTP ALG:

```
map fei0 10.0.0.0/8 to 0/32 proxy port 21 ftp/tcp
```

Proxy rules begin exactly like basic NAT or NAPT rules with the addition of the following elements:

- The proxy keyword.
- A specification of the trigger port that will cause ALG to be called— typically the well-known port for the service the ALG is meant to handle.
- A string identifying the ALG.
- The protocol that, in combination with the specified port, causes the ALG routine to be called.

The protocol specified in the rule must be the same as the protocol for which the ALG is registered. Proxy rules translate the packet's address and port in accordance with NAPT rules, but the new source port is allocated from the automatic port interval, which is set in the **IPNAT_AUTOPORT_START_INTERVAL** and **IPNAT_AUTOPORT_END_INTERVAL** components. For more information on these NAT system variables, see NAT Configuration Parameters.

The sample rule above pertains to all packets going out on interface fei0 with a source address matching the network 10.0.0.0/8. The rule instructs NAT to replace the source address in such packets with the interface address of fei0—but only when the destination port is equal to 21 and the protocol is TCP. The rule also instructs NAT to pass the packet to the ALG routine defined by the identifier ftp.

In some rare cases, the proxy rule may need to disable NAPT, because sometimes the protocol does not allow the source port to be changed. For this purpose the nonapt keyword can be added after to the proxy rule. The IKE protocol may require that the source port is not changed. The rule below shows an example of the nonapt keyword.

```
map eth0 0/0 to 0/32 proxy port 500 ipsec/udp nonapt
```

You can also add ALG rules for protocols other than TCP or UDP. For example:

```
map fei0 10.0.0.0/8 to 0/32 proxy port 0 ipsec/esp
```

This rule enables the IPsec proxy for all ESP encapsulating security payload (ESP) packets coming from the 10.0.0.0/8 network (RFC 2406). It instructs NAT to set the trigger port to 0 when the protocol is not TCP or UDP. ALG rules can be added for all protocols except ICMP and ICMPv6.

Some ALGs can handle bidirectional NAT as well as traditional NAT. To enable an ALG for bidirectional NAT, add the keyword proxy, followed by the identifier of the ALG at the end of the redirection rule. The FTP ALG is an example of an ALG that can handle both traditional NAT and bidirectional NAT. The following example shows how to configure bidirectional NAT to an FTP server on the private network and enable the FTP ALG at the same time.

```
rdr fei0 195.42.198.1 port 21 to 10.0.0.1 port 21 tcp proxy ftp
```

The proxy trigger port is the port to the left of the address join string (to), and the proxy protocol is TCP. You can also enable ALG support for redirection rules for protocols other than TCP or UDP, as shown in the following example:

```
rdr fei0 195.42.198.1 port 0 to 10.0.0.1 port 0 gre proxy pptp
```

Some protocols need ALGs to enable NAT-PT. For example, you may need to translate embedded IPv6 addresses to IPv4 addresses or vice versa. To configure NAT-PT with ALG support, add a NAT-PT rule that specifies the proxy keyword, the ALG identifier, and the protocol in the same manner as traditional NAT proxy rules. The following example enables the DNS ALG for NAT-PT. DNS typically runs over UDP and uses port 53.

```
pt fei1 ::/0 to 195.42.198.5 proxy port 53 dns/udp
```

Wind River NAT ALGs on page 27 summarizes the ALGs provided by Wind River NAT.

Table 1. Wind River NAT ALGs

ALG	ID	Protocol	Supports Outbound (Traditional NAT)	Supports Inbound (Bidirectional NAT)	Supports NAT-PT
DNS	"dns"	udp	No	No	Yes
FTP	"ftp"	tcp	Yes	Yes	Yes

ALG	ID	Protocol	Supports Outbound (Traditional NAT)	Supports Inbound (Bidirectional NAT)	Supports NAT-PT
H.323	"h323"	tcp	Yes	Yes	No
IPsec Passthrough	"ipsec"	udp	Yes	No	No
	"ipsec"	esp	Yes	No	No
PPTP Passthrough	"pptp"	tcp	Yes	No	No
	"pptp"	gre	Yes	No	No
Port triggering	"example_tcp"	tcp	Yes	No	No
	"example_udp"	udp	Yes	No	No
SIP	"sip"	udp	Yes	No	No

Note: Port triggering entries are examples only. Configure actual entries in accordance with your application.

3. ICMP ALG Operation

The ICMP ALG is built into the Wind River NAT software itself. There is no need to create a custom ALG.

4. DNS ALG Operation

The domain name system (DNS) is commonly used on the Internet to match a host name to an Internet address and vice versa. The DNS protocol is described in RFC 1034 and RFC 1035. Extensions to DNS required for IPv6 are described in RFC 1886 and RFC 2874.

Generally, the DNS protocol is NAT-friendly, which means that no ALG is required for NAT operation in basic mode or NAPT mode when the name server is placed on the outside of the NAT. However, when running in bidirectional NAT mode with the name server located on the private side of NAT, a DNS ALG is required. This mode is not supported by the DNS ALG. Instead, locate the name server outside of the NAT in this configuration.

A second scenario where a DNS ALG is required is for NAT-PT when a private IPv6 network is behind a NAT-PT router and the name server is located on the public IPv4 network. Wind River NAT supports this configuration, which enables a local IPv6 network to communicate transparently with IPv4 servers on the Internet, using their host names.

When an IPv6 host on the inside of the NAT looks up the address of a corresponding host name, it queries for AAAA records. Normally the name server has no entry for AAAA records because it is likely to be an IPv4-only server. Therefore the DNS ALG translates the AAAA record in the query to an A record. When the answer comes back, it translates the A record back to an AAAA record. The ALG matches incoming responses to outgoing requests by tracking the transaction ID of the DNS messages.

The DNS ALG also supports reverse name lookups. When a DNS client queries for the name corresponding to a certain address, it uses PTR records. The DNS ALG translates the PTR record so that it contains the IPv4 part of the IPv6 address and changes the zone

name, which can be either ip6.int or ip6.arpa to in-addr.arpa. When the answer comes back it restores the IPv6 address and zone so that it corresponds to the original requests.

The DNS ALG is included at compile time by enabling the #define **IPNET_USE_NAT_DNS_ALG** macro at compile time. It can be used only with NAT-PT. The macro is located in the following file:

installDir **/vxworks-7/pkgs/net/ipnet/coreip-***x.x.x.x* **/src/ipnet2/config/ipnet_c onfig.h**

5. FTP ALG Operation

FTP is one of the most popular applications for remote file transfer. For an FTP application to work with NAT, it requires an ALG to monitor the control session payload to determine the ensuing data session parameters.

The optional FTP ALG component in NAT fully supports the required functionalities specified in RFC 3022 (*Traditional IP Network Address Translator (Traditional NAT): 4.4 FTP support*).

The FTP ALG also supports the EPRT and EPSV command extensions, as specified in RFC 2428 and is capable of handling NAT-PT, as described in RFC 2766 (*Network Address Translation - Protocol Translation (NAT-PT): 6.2, Payload modifications for V6 originated FTP sessions*).

The FTP ALG is included at compile time by enabling the #define **IPNET_USE_NAT_FTP_ALG** macro at compile time. It can be used with traditional NAT, bidirectional NAT, and NAT-PT. The macro is located in the following file:

installDir **/vxworks-7/pkgs/net/ipnet/coreip-***x.x.x.x* **/src/ipnet2/config/ipnet_c onfig.h**

6. H.323 ALG Operation

H.323 is a standard published by the International Telecommunication Union— Telecommunication Standardization sector (ITU-T) specifying multimedia video conferencing on packet-switched networks such as LANs and the Internet.

The standard comprises a set of protocols for voice, video, and data conferencing on packet-switched networks.

> Note: A discussion of H.323 architecture and its protocols is outside the scope of this manual. Information about the H.323 standard is largely available on the Internet. For an in-depth understanding of this standard, refer to the Kumar, Korpi, and Sengodan book noted in Additional Documentation.

H.323 is complex, uses dynamic ports, and includes multiple UDP and TCP streams. However, the H.323 ALG component must handle only two associated protocols: H.225 and H.245.

H.225

This protocol defines the procedures and signaling between two endpoints for setting up and releasing a call to TCP port 1720. Note that port 1720 is the well-known call-signaling port for H.323.

H.245

This protocol defines procedures and signaling between two endpoints in order to exchange capabilities and control media streams.

The H.225 payload contains address and port number fields for setting up calls and preparing for the H.245 connection. Similarly, the H.245 payload includes various address and port fields for creating media control and data streams. Since both of these protocols sit above the transport layer, a specialized ALG is required to translate the addresses and port numbers in the payloads. For this reason, the H.323 ALG actually comprises two ALGs: H.225 ALG and H.245 ALG. Both of these ALGs are registered to NAT. The H.225 ALG

is registered at H.323 ALG initialization and is associated with TCP port 1720. The H.245 ALG is registered during the H.225 session (that is, during call setup) because there is no well-known port associated with the H.245 protocol, which is an ephemeral port lasting only for the duration of the video conferencing session.

The H.323 ALG component does not actually parse both the H.225 and the H.245 payloads. The messages of both these protocols are encoded in ASN.1. Rather than employing the ASN.1 decoder within the ALG, the ALG does a byte-by-byte search for the IP address and TCP/UDP port number in the payload. (Fortunately, the port number always follows immediately after the address.) By interacting with NAT through the NAT API, the ALG correctly processes the packet according to its direction of flow, original address and port number, and translated address and port number. The ALG creates mappings for all of the negotiated control and data streams (mostly UDP streams, except for the T.120 stream with its well-known TCP port 1503).

The H.323 ALG component has been successfully tested with the Microsoft NetMeeting application. Because the ALG does not have the ASN.1 decoder to interpret the H.225 and H.245 messages, it is not guaranteed to always work with other applications using the H.323 protocol.

The H.323 ALG is included at compile time by enabling the #define **IPNET_USE_NAT_H323_ALG** macro at compile time. It can be used with traditional NAT and bidirectional NAT but not with NAT-PT. The macro is located in the following file:

installDir **/vxworks-7/pkgs/net/ipnet/coreip-***x.x.x.x* **/src/ipnet2/config/ipnet_c onfig.h**

7. IPsec Passthrough ALG Operation

IPsec is a set of protocols developed by the IETF to support the secure exchange of packets at the IP layer.

IPsec has been deployed widely to implement virtual private networks (VPNs).

IPsec supports two encryption modes: transport and tunnel. Transport mode encrypts only the data portion (payload) of each packet but leaves the header untouched. The more secure tunnel mode encrypts both the header and the payload. On the receiving side, an IPsec-compliant device decrypts each packet.

The IPsec/IKE passthrough ALG allows IPsec VPN traffic to pass through a router using NAT. This passthrough service is limited to IPsec in ESP tunnel mode only (RFC 2406). The passthrough ALG performs translation on both Internet Security Association and Key Management Protocol (ISAKMP, RFC 2408) and ESP packets. ISAKMP packets are tracked and translated by using the cookies present in those messages. ESP packets are tracked and translated by using the security parameter index present in those messages. This allows for multiple IPsec connections to pass through the NAT router.

The IPsec ALG is included at compile time by enabling the #define **IPNET_USE_NAT_IPSEC_ALG** macro at compile time. It can be used only with traditional NAT. The macro is located in the following file:

installDir /vxworks-7/pkgs/net/ipnet/coreip-*x.x.x.x* /src/ipnet2/config/ipnet_c onfig.h

8. PPTP Passthrough ALG Operation

The Point-to-Point-Tunneling Protocol (PPTP) is a networking technology that supports multi-protocol VPNs, enabling remote users to access corporate networks securely across point-to-point protocol (PPP)-enabled systems—that is, to dial into a local Internet service provider to connect securely to their corporate network through the Internet.

PPTP enables a low-cost, private connection to a corporate network through the public Internet. This capability is particularly useful for people who work from home or people who travel and must access their corporate networks remotely to check email or perform other activities.

The PPTP passthrough ALG allows PPTP VPN traffic to pass through a router using NAT (NAPT mode only). It performs translation on both PPTP control and generic routing encapsulation packets (GRE, RFC 1701). Both types of packets are tracked and translated by using the call IDs present in those messages. This functionality allows multiple PPTP connections to pass through the NAT router.

The PPTP ALG is included by enabling the #define **IPNET_USE_NAT_PPTP_ALG** macro at compile time. It can be used only with traditional NAT. The macro is located in the following file:

installDir `/vxworks-7/pkgs/net/ipnet/coreip-`*x.x.x.x* `/src/ipnet2/config/ipnet_c onfig.h`

9. SIP ALG Operation

According to RFC 3261, the session initiation protocol (SIP) is an application-layer control protocol that you can use to establish, modify, and terminate multimedia sessions (conferences) such as Internet telephony calls.

Wind River NAT provides a means to handle RFC 3261 messages.

The SIP ALG supplied with Wind River NAT is a minimal implementation that applies only to SIP over UDP. It allows simple SIP UDP sessions to pass through a NAT device. If a SIP message originates behind a Wind River NAT device (on the private network), it can use its private address and port number to provide contact information for the remote host. (This information is embedded in the various headers.) The ALG handles all necessary address and port translations for SIP request messages and SIP response messages, provided that they originate behind the Wind River NAT device (on the private network).

This ALG is enabled by a macro at compile time, and once enabled, requires no further initialization. To implement SIP, simply write a rule that specifies the name of the ALG and the protocol. The default port for SIP is 5060, but you can designate any UDP port in the NAT rule.

The SIP ALG is included by enabling the #define **IPNET_USE_NAT_SIP_ALG** macro at compile time. It can be used only with traditional NAT. The macro is located in the following file:

installDir **`/vxworks-7/pkgs/net/ipnet/coreip-`***x.x.x.x* **`/src/ipnet2/config/ipnet_c onfig.h`**

10. Port Triggering

The port trigger ALG lets you dynamically open inbound ports to external connections, based on outbound traffic.

The ALG does not do any actual parsing of the packet payload. Instead, it adds a mapping to a host on the internal network when an outgoing connection is made to the port on which the ALG is registered. The inbound connection is sent to the same private host that caused the trigger to be hit. The trigger ALG is configured by the cookie parameter in the call to the routine ipnet_nat_add_proxy( ). The cookie has the following type:

```
typedef struct Ipnet_nat_trigger_struct
{
Ip_u8 protocol;
Ip_u16 portlo;
Ip_u16 porthi;
Ip_u32 timeout;
Ipnet_nat_proxy_func func;
void *cookie;
}
Ipnet_nat_trigger;
```

The protocol parameter must be either **IP_IPPROTO_UDP** or **IP_IPPROTO_TCP** and specify the protocol for which the port shall be opened. The portlo and porthi parameters are the triggered ports and specify the port range that will be opened. If only one port will be opened, both must have the same value. The timeout parameter specifies how long the port will remain open after the trigger port has been hit. The func and cookie parameters are optional and are used to specify an ALG that is called when packets go through the triggered ports. Below is an example configuration structure that opens the SSH port (22) for 120 seconds after the trigger port has been hit.

```
    Ipnet_nat_trigger example_trigger = {IP_IPPROTO_TCP,
                                         22,
                                         22,
                                         120,
                                         IP_NULL,
                                         IP_NULL};
```

The timer is reset each time there is traffic to either the trigger port or the triggered ports.

To include the trigger ALG, enable the #define **IPNET_USE_NAT_TRIGGER_ALG** macro at compile time. This ALG can be used only with traditional NAT. The macro is located in the following file:

installDir **/vxworks-7/pkgs/net/ipnet/coreip-***x.x.x.x* **/src/ipnet2/config/ipnet_c onfig.h**

11. Writing a Custom ALG

The NAT module provides an interface for writing custom ALGs.The list of protocols used on the Internet is constantly evolving, which calls for a published API that can be used to register new ALGs with the NAT. This section describes the methods and types available for this purpose.

Adding Your ALG

Before the ALG can be used by any NAT rules, it must be added to the NAT module. Use the API routine ipnet_nat_add_proxy( ). This routine registers the ALG with the NAT module so that it can accept rules that refer to the ALG.

The routine accepts four parameters, of which the first three are mandatory. The mandatory parameters are:

- The name of the ALG.
- The protocol to which the ALG applies
- The actual ALG routine that is called when a NAT rule is hit by a packet.

The same ALG can be registered for several protocols. ALGs can be added for all protocols but ICMP and ICMPv6.

The last parameter is optional and is used as a cookie that is provided in the call to the ALG routine. The cookie can be used for configuring the ALG as shown in Port Triggering, p.40.

To remove an ALG from the NAT module, verify that no NAT rules refer to it. Then call ipnet_nat_remove_proxy( ).

The following example illustrates the addition of a new ALG:

```
errval = ipnet_nat_add_proxy("my_alg", "tcp", my_custom_alg, IP_NULL);
```

Adding the NAT Rule

Once the ALG has been added to the NAT, you can add a NAT rule that refers to it. The rule must specify the same name and protocol for which the ALG is registered or the addition of the rule fails. Rules referring to an ALG must be added before other NAT rules. Otherwise, packets may match a different rule and the ALG will never come into operation. Below is an example of a NAT rule specifying an ALG. The ALG called **my_alg** is called when a private host makes an outgoing TCP connection to port 12345. All further traffic, incoming or outgoing, on this port also calls the ALG.

```
map fei0 0/0 to 0/32 proxy port 12345 my_alg/tcp
```

Writing the ALG Routine

The ALG routine is the piece of code that performs the task the ALG is meant to accomplish. ALGs are typically used for:

- Scanning application data for IP addresses in order to replace them with the NAT-translated address and port.
- Opening ports or protocols to incoming traffic.
- Providing NAT functionality when a protocol without ports needs to be translated.

The ALG routine must have the following type:

```
typedef int (* Ipnet_nat_proxy_func) (Ip_u8 *newhdr,
   Ip_u8 *appdata,
   int *applen,
   int growspace,
   Ipnet_nat_proxy_param *param,
   Ip_u8 **newdata);
```

The return value is an integer and must have one of the following values: 1, 0, or a negative value.

- A value of 1 means the ALG has modified application data.
- A value of 0 means application data is unchanged.
- A negative value means that the packet will be dropped by the NAT module.

The ALG routine must have six arguments:

- *newhdr*
- *appdata*
- *applen*
- *growspace*
- *param*
- *newdata*

The above parameters are defined as follows:

newhdr

This parameter is a pointer to a memory location where the ALG can optionally create a new IP header. NAT normally creates a new header, so this step may not be required. However, certain protocols are not based on UDP or TCP and therefore do not have any ports that can be used by the NAT module to check incoming packets against mappings and direct them to the correct private network host. ALGs for these protocols attempt to use another protocol field for this purpose. Such ALGs include IPsec, which uses the SPI field, and PPTP, which uses the call ID field.

WNDRVR

appdata

This parameter is a pointer to the protocol's application data. This pointer indicates the location where the ALG typically scans for embedded IP addresses and ports.

applen

This parameter is a pointer to the protocol's application data length. If the ALG modifies the application data so that the length is changed, the new length must be reflected in the parameter.

growspace

This parameter tells the ALG how much (in bytes) the application data can grow before a new buffer must be created.

param

This parameter is a pointer to some useful proxy parameters. The proxy parameters represented by param have the following type:

```
typedef struct Ipnet_nat_proxy_param_struct
{
    Ipnet_nat_proxy_tuple tuple;
    Ip_u32  nat_addr;
    Ip_u16  nat_port;
    void    *mapping;
    Ip_bool inbound;
    Ip_bool incoming;
    Ip_bool natpt;
    Ip_u32  prefix[3];
    void    *cookie;
    Ip_u32  fragid;
    Ip_u16  fragoff;
    Ip_u8   fragmf;
}
Ipnet_nat_proxy_param;
```

Where the proxy parameters are as follows:

tuple

A pointer to the proxy tuple structure. It includes information about the addresses, ports, and protocol that created the NAT mapping that caused the call to the ALG. See below for a detailed description of the tuple structure.

nat_addr and nat_port

These parameters are the new source address and source port the packet is given after it has been translated by NAT.

mapping

A pointer to the NAT mapping that caused the call to the ALG. It is provided in the proxy parameters because some of the functions available for ALGs require the parent mapping to be given as argument.

inbound

This parameter is a boolean value that tells the ALG if the mapping that caused the call to the ALG was created by an inbound packet (bidirectional NAT) or an outbound packet.

incoming

A boolean that tells the ALG if the packet that caused the call to the ALG was an incoming packet.

natpt

A boolean that tells the ALG if the mapping that caused the call to the ALG was created by a NAT-PT rule.

prefix

This parameter is set to the IPv6 prefix used by NAT-PT.

cookie

The cookie that was set when adding the ALG.

fragid, *fragoff*, and *fragmf*

These parameters are used to indicate if the packet is a fragment or not. fragid is the fragment identifier, fragoff is the fragment offset, and fragmf is the more fragments bit (the bit that indicates whether more fragments exist).

The proxy tuple structure has the following format:

```
typedef struct Ipnet_nat_proxy_param_struct
   {
   Ipnet_nat_proxy_tuple tuple;
   Ip_u32 nat_addr;
   Ip_u16 nat_port;
   void *mapping;
   Ip_bool inbound;
   Ip_bool incoming;
   Ip_bool natpt;
   Ip_u32 prefix[3];
   void *cookie;
   Ip_u32 fragid;
   Ip_u16 fragoff;
   Ip_u8 fragmf;
   }
   Ipnet_nat_proxy_param;
```

Where:

private_port

The port used by the host on the private side of the NAT. This value will be 0 for protocols other than UDP, TCP, or ICMP echo.

public_port

The port used by the host on the public side of the NAT. This value will be 0 for protocols other than UDP, TCP, or ICMP echo.

private_address

The address of the host on the private side of the NAT. public_address The address of the host on the public side of the NAT.

protocol

The IP protocol that created the mapping.

newdata

This parameter is a pointer to a pointer to a new application data buffer in case the packet grows more than what is possible without creating a new buffer (as indicated by *growspace*). Use ipcom_malloc( ) to allocate a new application data buffer, which is automatically freed by the NAT module when the packet has been sent.

WNDRVR

Routine Available for ALGs

The following routines are available to you as the ALG writer. For more detailed information, see the reference entry for each routine.

ipnet_nat_proxy_add_mapping( )

Opens a port or protocol through the NAT to a host on the private network for which there is no bidirectional NAT rule.

ipnet_nat_proxy_set_mapping_timeout( )

Sets a timeout for the mapping that caused the call to the ALG.

ipnet_nat_proxy_get_time( )

Gets the elapsed time since boot.

ipnet_nat_proxy_timeout_schedule( )

Schedules a routine to be called in the future.

ipnet_nat_proxy_timeout_reschedule( )

Reschedules a previously scheduled routine.

ipnet_nat_proxy_timeout_cancel( )

Cancels a previously scheduled routine.

12. Sample Rule Sets with ALG Support

ALG support can be added to a variety of rule set scenarios. The examples below are used to illustrate some common rule sets.

NAT Router with ALG Support

```
map fei0 0/0 to 0/32 proxy port 21 ftp/tcp
map fei0 0/0 to 0/32 proxy port 1723 pptp/tcp
map fei0 0/0 to 0/32 proxy port 1720 h323/tcp
map fei0 0/0 to 0/32 proxy port 500 ipsec/udp nonapt
map fei0 0/0 to 0/32 proxy port 5060 sip/udp
map fei0 0/0 to 0/32 portmap tcp/udp 18000:18999
map fei0 0/0 to 0/32 icmpidmap icmp 19000:1999
map fei0 0/0 to 0/32
```

NAT Router with ALG Support and DMZ Host

```
map fei0 0/0 to 0/32 proxy port 21 ftp/tcp
map fei0 0/0 to 0/32 proxy port 1723 pptp/tcp
map fei0 0/0 to 0/32 proxy port 1720 h323/tcp
map fei0 0/0 to 0/32 proxy port 500 ipsec/udp
map fei0 0/0 to 0/32 proxy port 5060 sip/udp
map fei0 0/0 to 0/32 portmap tcp/udp 18000:18999
map fei0 0/0 to 0/32 icmpidmap icmp 19000:19999
map fei0 0/0 to 0/32
rdr fei0 195.42.198.1 port 22 to 195.42.198.1 port 22 tcp
rdr fei0 195.42.198.1 port 0 to 195.42.198.1 port 0 icmp
```

```
rdr fei0 195.42.198.1 port 1720 to 10.0.0.2 port 1720 tcp proxy h323
rdr fei0 195.42.198.1 port 0 to 10.0.0.1 port 0 ip
```

NAT-PT Router with ALG Support

```
pt fei1 ::/0 to 195.42.198.5 proxy port 21 ftp/tcp
pt fei1 ::/0 to 195.42.198.5 proxy port 53 dns/udp
pt fei1 ::/0 to 195.42.198.5 portmap tcp/udp 18000:18999
pt fei1 ::/0 to 195.42.198.5 icmpidmap icmp 19000:19999
pt fei1 ::/0 to 195.42.198.5
```


WNDRVR

6. WIND RIVER NAT KEYWORDS

1. Overview

This appendix provides reference information for the Wind River NAT keywords used to define NAT rules.

2. Rule Synopsis Reference

NAT Rule Synopsis

[@*index* **]** **{map|map-block|pt|pt-block}** *interface private_source_address* **[**/*mask* **]** **to** *public_source_address* **[**/*mask* **]**
[[portmap {tcp|udp|tcp/udp} low_port_number:high_port_number | auto:size_of_range] |
[icmpidmap icmp *low_port_number:high_port_number* **]**

[proxy port *port_number proxyname* /*protocol* **][nonapt]** |
[dstmap *destination_address* **]]**

NAT Redirect Rule Synopsis

[@*index* **]** **rdr** *interface destination_address* **[**/*mask* **]** **port** *port_number to private_host_address* **port** *port_number* **[**protocol **]** **[proxy** *proxyname* **]**

3. Keyword Reference

to

Description

A string used to indicate a mapping between a private address and a public address. **Synopsis**

Synopsis

{map|map-block|pt|pt-block} *interface private_source_address* **to** *public_source_address*

#

Description

Precedes a comment.

Synopsis

*comment*

map

Description

Specifies a public source address with which to replace the private source address on outgoing packets on the specified interface. Also establishes a correlation, or mapping, between the original private source address and public source address in the translated packet. This mapping is used to route incoming packets received in response to the outgoing packet to the correct private network host. Used to configure basic NAT.

Synopsis

`map` *interface private_source_address* **`to`** *public_source_address*

interface

The interface on which the outgoing packet is transmitted.

private_source_address

Can be a private host address, an address space on the private network, or a wildcard signifying any private address (0/0).

public_source_address

Can be the public address of the gateway or the address of the specified interface.

dstmap

Description Translates the destination address when sending a packet through NAT.

Synopsis

`map` *interface private_source_address* **`to`** *dstmap destination_addr*

interface

The interface on which the outgoing packet is transmitted.

private_source_address

Can be a private host address, an address space on the private network, or a wildcard signifying any private address (**0/0**).

destination_address

The IP address that the destination address will be changed to.

For example, to translate any source address to a NAT external address and the destination address to 192.168.0.2, use:

```
map fei0 0/0 to 0/32 dstmap 192.168.0.2
```

The printouts of a nat -m (list states) command are similar to the following:

```
10.7.1.1:40000 -> 10.7.2.10:30000 proto udp, (10.7.2.1:40000), dstmap: 10.7.2.3, state EST/EST
, expire in 60 s
```

icmpidmap

Description

For ICMP echo requests or replies, instructs NAT to perform address and port translation based on the identifier field of the ICMP echo header. Used in conjunction with the **map** keyword to configure NAPT.

Synopsis

`map` *interface private_source_address* **`to`** *public_source_address* **`icmpidmap icmp`** *low_port_number:high_port_number*

interface

The interface on which the outgoing packet is transmitted.

private_source_address

Can be a private host address, an address space on the private network, or a wildcard signifying any private address (**0/0**).

public_source_address

Can be the public address of the gateway or the address of the specified interface.

low_port_number and high_port_number

Can be any port number.

map-block

Description

Specifies a public source address with which to replace the private source address on outgoing packets on the specified interface. Differs from **map** in that each private address is assigned a unique public address. Also establishes a correlation, or mapping, between the original private source address and public source address in the translated packet. This mapping is used to route subsequent incoming packets to the correct private network host. Used to configure basic NAT.

Synopsis

`map-block` *interface private_source_address* **`to`** *public_source_address*

private_source_address

Can be a private host address, an address space on the private network, or a wildcard signifying any private address (**0/0**).

public_source_address

A public address space from which a unique public address is substituted for the specified *private_source_address*.

nonapt

Description

Disables NAPT when a protocol does not allow the source port to be changed. In such cases, append the **nonapt** keyword to the **proxy** rule. The IKE protocol may require that the source port is not changed. For further information on the specific usage of this keyword, see *proxy on page 41*.

port

Description

Specifies the port number used in proxy and rdr rules. For further information on the specific usage of this keyword, see *proxy on page 41*, and *rdr on page 43*.

portmap

Description

Specifies the source port translation for outgoing packets that meet the specified parameters for source address, interface, and protocol. Used in conjunction with the **map** keyword to configure NAPT.

Synopsis

`map` *interface private_source_address* **`to`** *public_source_address* **`portmap`** *protocol low_port_number* **:***high_port_number* **`| auto`**:*size_of_range*

interface

The interface on which the outgoing packet is transmitted.

private_source_address

Can be a private host address, an address space on the private network, or a wildcard signifying any private address (**0/0**).

public_source_address

Can be the public address of the gateway or the address of the specified interface.

protocol

Can be tcp, upd, or tcp/udp.

low_port_number and high_port_number can be any port number.

auto and *size_of_range*

Specify a range and the system decides the start and end port. This is done by using the **auto** keyword instead of the starting port. Thus, set auto with the ending port being the size of the range. For example: **auto:5000** gives you a range of 5000 ports where the system decides what the start and ending ports will be.

proxy

Description

Used in conjunction with the keywords **map**, **map-block**, **pt**, **pt-block**, or **rdr** to configure an ALG.

Synopsis

`{map|map-block|pt|pt-block}` *interface private_source_address* **`to`** *public_source_address* **`proxy port`** *port_number alg* **`[`** */protocol* **`] [nonapt]`**

interface

The interface on which the outgoing packet is transmitted.

private_source_address

Can be a private host address, an address space on the private network, or a wildcard signifying any private address (**0/0**).

public_source_address

A public address space from which a unique public address is substituted for the specified *private_source_address* .

port_number

The trigger port that causes the ALG to be called (typically the well known port for the service the ALG is meant to handle). The new source port is allocated from the automatic port interval set in the Workbench kernel components **IPNAT_AUTOPORT_START_INTERVAL** and **IPNAT_AUTOPORT_END_INTERVAL**. For further information on these variables, see NAT Configuration Parameters.

alg

The identifier of the ALG.

protocol

The protocol that, in combination with the specified port, causes the ALG to be called. *protocol* must be the same as the protocol for which the ALG is registered.

In some rare cases, it may be necessary for the proxy rule to disable NAPT when the protocol does not allow the source port to be changed. For this purpose, append the **nonapt** keyword to the proxy rule. The IKE protocol may require that the source port is not changed.

For bidirectional NAT, a **port** keyword specifying the proxy trigger port precedes the to string. A second **port** keyword follows the private host address, specifying the port on the private host to which the packet is to be redirected.

rdr *interface destination_address* **port** *port_number* **to** *private_host_address* **port** *port_number protocol*
proxy *proxyname*

pt

Description

Specifies an IPv4 public source address with which to replace the IPv6 private source address on outgoing packets on the specified interface. Also establishes a correlation, or mapping, between the original private source address and public source address in the translated packet. This mapping is used to route incoming packets received in response to the outgoing packet to the correct private network host. Used to configure NAT-PT.

Synopsis

pt *interface private_source_address* **to** *public_source_address*

interface

The incoming interface on the IPv6 side of the gateway.

private_source_address

The IPv6 address of the private host transmitting the packet. Can optionally include an IPv6 prefix of up to 128 bits.

public_source_address

The translated source address in IPv4 format of the outgoing packet.

pt-block

Description

Specifies an IPv4 public source address with which to replace the IPv6 private source address on outgoing packets on the specified interface. Differs from pt in that each private address is assigned a unique public address. Also establishes a correlation, or mapping, between the original private source address and public source address in the translated packet. This mapping is used to route subsequent incoming packets to the correct private network host. Used to configure NAT-PT.

Synopsis

pt-block *interface private_source_address* **to** *public_source_address*

interface

The incoming interface on the IPv6 side of the gateway.

private_source_address

The IPv6 address of the private host transmitting the packet. Can optionally include an IPv6 prefix of up to 128 bits.

public_source_address

The translated source address in IPv4 format of the outgoing packet.

public_source_address

A public address space from which a unique IPv4 public address is substituted for the specified IPv6 *private_source_address* .

rdr

Description

Redirects incoming packets that meet the specified parameters for destination address, interface, port, and protocol to the specified private host and port. Used to configure bidirectional NAT, a DMZ host, or an ALG.

Synopsis

rdr *interface destination_address* **[** */mask* **]** **port** *port_number* **to** *private_host_address* **port** *port_number* **[***protocol* **]** **[pro xy** *proxyname* **]**

interface

The interface on which the incoming packet is received.

destination_address

The destination address specified in the packet of the incoming packet.

port_number

Appears twice in an rdr rule. In the first instance, *port_number* is the destination port number of the incoming packet. In the second instance, *port_number* is the port number of the private host to which the packet is redirected. To specify any port, use 0.

WNDRVR

private_host_address

The address of the private host to which the incoming packet is redirected.

protocol

The protocol of the incoming packet, which can be gre, icmp, ip, tcp, udp, or tcp/udp. The protocol number can also be used.

to

Description

A string used to indicate a mapping between a private source address and a public source address.

Synopsis

`{map|map-block|pt|pt-block}` *interface private_source_address to public_source_address*

WNDRVR

7. NAT SHELL COMMAND REFERENCE

1. About the NAT Shell Command

Wind River NAT provides a shell command that you can use to enable and disable NAT support.

You can also use this shell command to modify NAT. For example, you can use the command to create NAT rules.

To include support for the nat shell command in your project, you must include the **INCLUDE_IPNAT_CMD** component in your VIP.

2. NAT Shell Command Options

Name

nat – enable, disable, and modify NAT.

Synopsis

`nat [-silent] {`*rule* **`| -f`** *filename* **`| -r`** *rule* **`| -n`** *rule* **`| -p | -s | -l | -m | -C | -D | -E | -F | -V | -Z }`**

Description

The nat command is used to add or remove NAT rules and to display NAT statistics. The command runs on a target shell.

> Note: To run this command, you must switch to the command interpreter shell before running the nat command. Type cmd at the command prompt. Then run the nat command.

The shell command options are as follows:

-silent

Suppress error output. This option is required for shell commands that are automatically executed at system startup.

-f

Add rules from the specified file.

-r

Remove the specified rule.

-n

Check syntax for the specified rule.

-p

Display loaded ALGs.

WNDRVR

-s

Display NAT statistics.

-l

Display rules.

-m

Display mappings.

-C

Clear active NAT mappings.

-E

Enable NAT.

-D

Disable NAT.

-F

Flush all rules and mappings.

-V

Show NAT version.

-Z

Clear NAT statistics.